Tutorial Worksheet 2

AI 501: Mathematics for Artificial Intelligence | Fall 2026

Problem 1

Let

$$A = \begin{pmatrix} 1 & 2 & 1 \\ -2 & -3 & 1 \\ 3 & 5 & 0 \end{pmatrix}.$$

(a) Find $\text{rank}(A)$.

(b) Give a basis for the column space $\mathcal{C}(A)$ and describe it geometrically.

(c) Find a basis for the null space $\mathcal{N}(A)$ and verify the rank–nullity theorem.

Solution:

(a) $R_2 + 2R_1 \to (0, 1, 3)$ and $R_3 - 3R_1 \to (0, -1, -3)$; then $R_3 + R_2 \to (0, 0, 0)$:

$$\begin{pmatrix} 1 & 2 & 1 \\ 0 & 1 & 3 \\ 0 & 0 & 0 \end{pmatrix}.$$

There are two pivots, so $\text{rank}(A) = 2$.

(b) The pivot columns are 1 and 2, so a basis is $\{(1, -2, 3)^T, (2, -3, 5)^T\}$. $\mathcal{C}(A)$ is a plane through the origin in $\mathbb{R}^3$ (the "rank 2 $\Rightarrow$ columns lie on a plane" case).

(c) From the echelon form, $x_2 = -3x_3$ and $x_1 = -2x_2 - x_3 = 5x_3$, so

$$\mathcal{N}(A) = \text{span}\{(5, -3, 1)^T\}.$$

Check: $5 - 6 + 1 = 0$, $-10 + 9 + 1 = 0$, $15 - 15 + 0 = 0$. Rank–nullity: $2 + 1 = 3$ columns.

Problem 2

The following is the confusion matrix of a classifier that detects the digit 5 on 60,000 MNIST images:

$$\begin{pmatrix} 53892 & 687 \\ 1891 & 3530 \end{pmatrix}.$$

Rows are the *actual* class and columns the *predicted* class, both in the order (not-5, 5). This is the transpose of the lecture's layout.

(a) Identify TP, FP, FN, and TN.

(b) Compute precision, recall, and F_1.

(c) Compute the arithmetic mean (AM) and geometric mean (GM) of precision and recall. Verify $\text{HM} \leq \text{GM} \leq \text{AM}$ and explain why F_1 uses the harmonic mean.

(d) A colleague says: "This model has about 96% accuracy, so it is an excellent 5-detector." Compute the model's accuracy and the accuracy of a classifier that always predicts "not 5." Explain what is wrong with the colleague's reasoning.

Solution:

(a) $TN = 53{,}892$, $FP = 687$, $FN = 1{,}891$, $TP = 3{,}530$.

(b) $\text{Precision} = \dfrac{3530}{4217} \approx 0.837$; $\quad \text{Recall} = \dfrac{3530}{5421} \approx 0.651$; $\quad F_1 = \dfrac{2TP}{2TP + FP + FN} = \dfrac{7060}{9638} \approx 0.733.$

(c) AM ≈ 0.744 and GM ≈ 0.738, so HM $= 0.733 \leq$ GM $\leq$ AM. The harmonic mean is pulled toward the smaller value, so a model cannot achieve a high F_1 by doing well on one metric and badly on the other.

(d) Model accuracy $= 57{,}422/60{,}000 \approx 0.957$. Always predicting "not 5" gives $54{,}579/60{,}000 \approx 0.910$ without detecting a single 5.

The reasoning fails because only about 9% of the images are 5s. Accuracy is dominated by the easy majority class, so a useless constant model already scores 91%. The high accuracy hides that the model misses about 35% of the real 5s (recall 0.651). For an imbalanced task, judge the model by precision, recall, or F_1, not accuracy; this is the same trap as the lecture's no-ball example.

Problem 3

Let p be the number of features, so each data point is a vector $(X_1, \ldots, X_p)$. Assume every feature is uniformly distributed on $[0, 1]$ and the features are independent.

To classify a test point, kNN uses only training points within 0.05 of the test point in *every* feature. For example, with $p = 1$ and a test point at $X = 0.6$, it uses training points with $X \in [0.55, 0.65]$. Ignore test points near 0 or 1, where this window would be cut off.

(a) With $p = 1$, what fraction of the training data falls inside the window?

(b) With $p = 2$ (the window is a 0.1×0.1 square), what fraction falls inside?

(c) With $p = 100$, what fraction falls inside?

(d) Based on (a)–(c), what goes wrong with kNN when p is large?

(e) Reverse the question: suppose the window is a cube of side L in every feature, and we want it to contain 10% of the training data. Find L for $p = 1$, 2, and 100.

Solution:

(a) The window has width 0.1, so 10%.

(b) Each feature is independently in range with probability 0.1, so $0.1 \times 0.1 = 1\%$.

(c) $0.1^{100} = 10^{-100}$: essentially no training points.

(d) The fraction of nearby points shrinks exponentially with p. To still find k neighbors, the window must grow until the "neighbors" are no longer close to the test point, so they are no longer similar and kNN's core assumption (similar inputs have similar labels) breaks down.

(e) We need $L^p = 0.1$, so $L = 0.1^{1/p}$: $p = 1$: $L = 0.1$; $p = 2$: $L \approx 0.316$; $p = 100$: $L \approx 0.977$. In 100 dimensions the window must span almost the whole range of every feature just to capture 10% of the data.